# AI Makes Code Cheap, Not Complexity: Lessons from Agent-Assisted Systems Engineering

**Mark Farkas**
*artalis.io*

**Abstract.** AI coding agents can increase implementation throughput, but throughput is not equivalent to system quality. This position paper compares four practitioner cases: Keel, a portable asynchronous transport substrate; Hull, a capability-secure execution substrate; bitnet.c and the planned Sail inference library; and Ralph, an experimental LP and MIP solver. The cases suggest that successful agent-assisted systems engineering depends on decomposition along semantic boundaries, explicit interfaces and invariants, and dense system-level feedback. Keel and Hull exhibit these properties. Inference exposes rising validation costs as architectural dimensions multiply. Ralph shows how abundant locally plausible code can fail to yield competitive global behavior. These observations motivate a qualitative model in which architecture and feedback, not code volume, bound system quality.

**Keywords:** AI-assisted programming, systems engineering, software architecture, validation, capability security

## 1. Introduction

Current coding agents substantially expand the amount of implementation one engineer can attempt. They generate routine code, tests, documentation, and integration work quickly. The resulting economic change is real, but its scope is narrower than "software becomes cheap." The central claim of this paper is: AI makes code cheap. It does not make complexity cheap.

System quality is not proportional to code volume. A useful qualitative relation is $Q = f(D, I, F)$, where $D$ is decomposition quality, $I$ is the strength of invariants and interfaces, and $F$ is the quality and density of system-level feedback. Generation throughput can accelerate work on all three, but cannot compensate for weak $D$, $I$, or $F$. This is a hypothesis derived from practitioner case studies, not a causal estimate from controlled experiments.

## 2. Comparative evidence

| Case | Architectural condition | Observed lesson |
|---|---|---|
| Keel | Protocol semantics are separated from readiness or completion execution and from POSIX, Winsock, lwIP, and UEFI providers. Lifecycle, ownership, cancellation, and backpressure contracts are explicit. | AI accelerated implementation and verification; expert decomposition established boundaries that protocol code cannot bypass. |
| Hull | Mechanism and policy are separated. Capabilities represent authority; paths, filesystem operations, blobs, execution, and application services have distinct responsibilities. | Coherence follows from explicit authority flow, static composition, and a small C enforcement core, with policy in Lua or JavaScript. |
| bitnet.c / Sail | Model family, tensors, quantization, backend, execution phase, memory layout, tokenizer, and sampling are separate dimensions. Sail limits families, formats, and target backends. | Decomposition aids tractability, but every dimension enlarges the integration and validation cross-product. |
| Ralph | The solver contains substantial presolve, scaling, simplex, factorization, cut, and branch-and-bound machinery plus comparison gates. | More local implementation did not reliably improve global behavior; feedback was sparse, delayed, and noisy. |

Keel's public contracts explicitly unify readiness and completion models while preserving transport-level lifecycle rules [1,2]. Hull's architecture places C capability checks between application code and operating-system resources, with manifest-driven policy and static composition [3]. In bitnet.c, registries and ownership rules separate model anatomy, quantization capabilities, backend-resident state, request-local sessions, tokenization, and sampling [4]. These repository structures corroborate the architectural observations; they do not by themselves prove that AI caused the outcomes.

Ralph is a useful negative experimental result, not a personal failure. Competitive LP and MIP solving depends on interacting numerical stability, presolve, scaling, basis management, simplex or barrier behavior, cuts, branching, primal heuristics, node selection, propagation, conflict analysis, recovery, and whole-search performance engineering. Ralph implements several of these mechanisms and includes direct GLPK and NETLIB comparison infrastructure [5]. The observation is that substantial generated implementation did not approach mature solver quality. One alternative interpretation is simply insufficient development time. Another is that solver engineering has unusually coupled objectives and low-density feedback. Both interpretations are consistent with the narrower conclusion: implementation throughput alone was not limiting.

## 3. Discussion

The cases distinguish three factors. First, AI strongly increases implementation throughput. Second, architectural coherence still depends on choosing semantic boundaries and maintaining dependency direction. Third, feedback must reveal system behavior at the level where quality is judged. Unit tests can validate a local parser or kernel, but cannot alone establish transport lifecycle correctness, authority confinement, exact inference behavior, or search-tree quality.

This comparison is subject to selection bias, author involvement, unequal project maturity, and differences in problem domain. Keel and Hull may also benefit from easier-to-state contracts, while Ralph confronts continuous numerical and heuristic tradeoffs. Thus the evidence supports a design proposition rather than a universal law: as coupling rises and feedback becomes delayed or noisy, faster code production increasingly risks accelerating structural failure.

## 4. Conclusion

Current coding agents expand implementation capacity, but amplify the architecture supplied to them. With strong decomposition, explicit invariants, enforceable boundaries, and rapid system-level feedback, they can help produce coherent systems such as Keel and Hull. Without those properties, locally reasonable contributions can accumulate into a globally incoherent system, as observed in Ralph. The limiting resource in large AI-assisted projects is therefore shifting from code production toward architectural judgment, coherent decomposition, and effective validation.

## 5. References

[1] artalis.io. "Keel repository and documentation." github.com/artalis-io/keel, accessed 2026-09-02.
[2] artalis.io. "libkeel documentation." libkeel.dev, accessed 2026-09-02.
[3] artalis.io. "Hull repository, architecture, and security documentation." github.com/artalis-io/hull, accessed 2026-09-02.
[4] artalis.io. "bitnet.c repository and architecture roadmap." github.com/artalis-io/bitnet.c, accessed 2026-09-02.
[5] artalis.io. "OTTO repository, Ralph source and GLPK comparison infrastructure." github.com/artalis-io/otto, accessed 2026-09-02.